

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 3

Duration: 1 Hour

Total Marks:30

Annexure A

Dry Run Evaluation

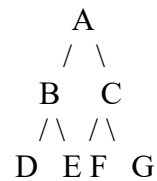
Code of Conduct:

I Nalluri Akhilesh Kumar(192365074) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Nalluri Akhilesh Kumar
Signature of the student:

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph:



Task: Starting from node **A**, perform a **Breadth-First Search (BFS)**. Complete the following table.

Iteration Queue Visited Node

1
2
3
4
5
6
7

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

```
File Edit Shell Debug Options Window Help Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/vanag/AppData/Local/Programs/Python/Python313/BFS.py =====
BFS Traversal:
Iteration Visited Node Queue
1 A ['B', 'C']
2 B ['C', 'D', 'E']
3 C ['D', 'E', 'F', 'G']
4 D ['E', 'F', 'G']
5 E ['F', 'G']
6 F ['G']
7 G []
>>>
```

```
File Edit Format Run Options Window Help
from collections import deque

# Graph represented as an adjacency list
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}

def bfs(graph, start):
    visited = set()
    queue = deque([start])

    visited.add(start)

    print("Iteration\tVisited Node\tQueue")

    iteration = 1

    while queue:
        node = queue.popleft()

        # Visit all unvisited neighbors
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

        print(f"{iteration}\t\t{node}\t\t{list(queue)}")
        iteration += 1

    print("BFS Traversal:")
    bfs(graph, 'A')
```

2. Depth-First Search (DFS) Using the same graph,



Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Perform **Depth-First Search (DFS)** starting from **A**.

Complete the table.

File Edit Format Run Options Window Help

```
# BFS Algorithm
def bfs(start, goal):
    queue = deque([[start, [start]]])
    visited = set([start])

    iteration = 1
    total_generated = 1

    while queue:
        current, path = queue.popleft()

        print(f"\nIteration {iteration}")
        print("Current State:")
        for row in current:
            print(row)

        if current == goal:
            print("\nGoal State Reached!")
            print("Total States Generated:", total_generated)
            print("Solution Length:", len(path) - 1)

            print("\nSolution Path:")
            for state in path:
                print()
                for row in state:
                    print(row)
            return

        print("\nQueue Size Before Expansion:", len(queue))

        for neighbor in get_neighbors(current):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, path + [neighbor]))
                total_generated += 1

        print("Queue Size After Expansion:", len(queue))

        iteration += 1
```

Ln: 50 Col: 0

FILE SHEET 3.13.3

Edit Shell Debug Options Window Help

Solution Length: 8

Solution Path:

(1, 3, 6)

(5, 0, 2)

(4, 7, 8)

(1, 3, 6)

(5, 2, 0)

(4, 7, 8)

(1, 3, 0)

(5, 2, 6)

(4, 7, 8)

(1, 0, 3)

(5, 2, 6)

(4, 7, 8)

(1, 2, 3)

(5, 0, 6)

(4, 7, 8)

(1, 2, 3)

(0, 5, 6)

(4, 7, 8)

(1, 2, 3)

(4, 5, 6)

(0, 7, 8)

(1, 2, 3)

(4, 5, 6)

(7, 0, 8)

(1, 2, 3)

(4, 5, 6)

(7, 8, 0)

1. Initial State :

1	3	6
5	0	2
4	7	8

Goal State :

1	2	3
4	5	6
7	8	0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.

Fill in the table.

Iteration Current State Queue Before Expansion Queue After Expansion

1
2
3
4
5

```
# BFS Algorithm
def bfs(start, goal):
    queue = deque([start, [start]])
    visited = set([start])

    iteration = 1
    total_generated = 1

    while queue:
        current, path = queue.popleft()

        print(f"\nIteration {iteration}")
        print("Current State:")
        for row in current:
            print(row)

        if current == goal:
            print("\nGoal State Reached!")
            print("Total States Generated:", total_generated)
            print("Solution Length:", len(path) - 1)

            print("\nSolution Path:")
            for state in path:
                print()
                for row in state:
                    print(row)

            return

        print("\nQueue Size Before Expansion:", len(queue))

        for neighbor in get_neighbors(current):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, path + [neighbor]))
                total_generated += 1

        print("Queue Size After Expansion:", len(queue))
```

Solution Length: 8

Solution Path:

(1, 3, 6)
(5, 0, 2)
(4, 7, 8)
(1, 3, 6)
(5, 2, 0)
(4, 7, 8)
(1, 3, 0)
(5, 2, 6)
(4, 7, 8)
(1, 0, 3)
(5, 2, 6)
(4, 7, 8)
(1, 2, 3)
(5, 0, 6)
(4, 7, 8)
(1, 2, 3)
(0, 5, 6)
(4, 7, 8)
(1, 2, 3)
(4, 5, 6)
(0, 7, 8)
(1, 2, 3)
(4, 5, 6)
(7, 0, 8)
(1, 2, 3)
(4, 5, 6)
(7, 8, 0)